



School of Future Tech

School Bus Tracking & Management System

Java Project Report

Daksh Ranjan Srivastava
Cohort - Jeff Bezos
Roll no - 150096725087
Subject - JAVA Programming

School Bus Tracking & Management System

Case Study 42 — Comprehensive Technical Documentation & Implementation Report

Student Name:	Daksh Ranjan Srivastava
Roll Number:	150096725087
Academic Program:	B.Tech Computer Science & Engineering (2025–29)
Course & Semester:	Java Programming Semester III
Case Study Topic:	Case Study 42: School Bus Tracking & Management System

1. Problem Statement

A school requires a system to manage buses, drivers, routes, students, and transportation schedules. The system should maintain student transportation records and allow administrators to monitor route assignments efficiently. In the absence of an automated management mechanism, institutions face severe operational vulnerabilities, including bus overcrowding, unassigned drivers, inefficient routing, lack of seat-allocation safety guarantees, and the inability to quickly audit transportation histories.

2. Objectives

- **Manage Buses and Drivers:** Maintain up-to-date registries of the institutional vehicle fleet (capacity, active driver) and certified commercial drivers (licenses, contact numbers).
- **Register Students for Transportation:** Capture student demographics, class details, and residential pickup addresses.
- **Manage Routes and Stops:** Establish designated school bus transit corridors and assign structured boarding stops.
- **Assign Students to Buses:** Allocate students to buses while enforcing maximum physical seating capacity atomically to prevent over-allocation.
- **Search and Sort Transportation Records:** Provide fast search capabilities (by student ID/name, bus number, route) and sorting algorithms (buses by capacity, routes by number).
- **Maintain Route and Student History:** Log all student allocation and reassignment actions into an audit history for institutional compliance.

3. Required Java Implementation Mapping

The project rigorously fulfills every concept mandated in the Case Study 42 curriculum. The following matrix summarizes what concept was used, how it was used, and why it was selected:

Java Concept	Application	How It Was Used	Why It Was Used
Classes & Objects	Bus, Driver, Student, Route, Stop	Encapsulated domain entities with private fields, getters, setters, and constructors.	Provides clean OOP modularity, separation of concerns, and data integrity.
Constructors	Object Initialization	Parameterized constructors initialize state and validate constraints upon instantiation.	Guarantees valid initial object states without incomplete records.
Array	Bus Seating Capacity	Fixed-size Student[] array inside Bus class representing physical seat slots.	Accurately models immutable physical vehicle seating boundaries with O(1) slot indexing.
ArrayList	Buses, Students, Drivers	ArrayList<Bus>, ArrayList<Student>, ArrayList<Driver> in Main.	Dynamic contiguous memory structure allowing constant-time append and indexed access.
LinkedList	Route Stops & History	LinkedList<Stop> inside Route, LinkedList<String> for travel history.	Maintains ordered sequential stops and efficient FIFO append operations for chronological logs.
HashMap	Bus Lookup Index	HashMap<Integer, Bus> mapping busNumber -> Bus instance.	Enables O(1) constant-time direct retrieval by bus number without scanning lists.
TreeMap	Route Directory	TreeMap<Integer, Route> indexed by routeNumber.	Automatically keeps routes sorted by natural key order (Red-Black tree backing).
CRUD Operations	All Domain Entities	Create, Read, Update, Delete implemented for Students, Buses, Drivers, Routes, Stops.	Provides comprehensive record management and synchronization across collections.
Searching	Record Retrieval	Sequential search by ID/Name, O(1) HashMap lookup by Bus #, TreeMap lookup by Route #.	Allows fast retrieval for administrators during student onboarding or queries.
Sorting	Fleet & Route Analytics	ArrayList sort using Comparator.comparingInt(Bus::getCapacity); TreeMap key traversal.	Allows administrative scheduling based on vehicle size and route hierarchy.

Swing GUI	BusTrack Pro Interface	JFrame with CardLayout, JTable, JTextField, styled dark mode panels.	Gives administrators a responsive desktop application with real-time feedback.
Exception Handling	Capacity Limits	Custom checked exception TransportationException thrown on capacity overflow.	Separates business rule violations from general runtime crashes; ensures atomic rollback.
Validation	Input Integrity Checks	Centralized validation in Main for names, ages, bus capacity, phone, and license strings.	Prevents corrupt, negative, empty, or duplicate records from entering data stores.

4. In-Depth System Architecture & Class Design

The system adheres to an Object-Oriented multi-tiered architecture. Below is an exhaustive breakdown of what each component does, how it operates, and why it is designed in this manner:

4.1 Entity Classes

A. Bus Class (Bus.java)

- **What it is:** The representation of a school bus vehicle in the transportation fleet.
- **How it is implemented:** Maintains private fields for busNumber (int), busName (String), capacity (int), driver (Driver), and an internal fixed-size array Student[] seats. It exposes methods addStudent(Student s), removeStudent(int studentId), isFull(), and getStudentCount().
- **Why this design:** Using a primitive array Student[] internal to Bus enforces physical capacity limits at the model layer. Rather than relying solely on high-level collection checks, the Bus object itself guarantees that passenger count never exceeds capacity.

B. Driver Class (Driver.java)

- **What it is:** The domain entity representing certified transportation staff.
- **How it is implemented:** Encapsulates driverId (int), driverName (String), phoneNumber (String), and licenseNumber (String) with validation ensuring non-null, non-empty values.
- **Why this design:** Separates personnel records from vehicle records. A Driver object can be assigned, unassigned, or reassigned across buses without modifying vehicle structural data.

C. Student Class (Student.java)

- **What it is:** The student passenger entity registered for transportation.
- **How it is implemented:** Stores studentId (int), studentName (String), age (int), className (String), address (String), and busNumber (int, initialized to -1 representing 'Not Assigned').
- **Why this design:** Decouples student enrollment from immediate bus allocation. A student can exist in the institution before being routed to a bus.

D. Route & Stop Classes (Route.java, Stop.java)

- **What they are:** Route models a school bus transit pathway; Stop represents physical boarding points.
- **How they are implemented:** Route maintains routeNumber (int), routeName (String), and a LinkedList<Stop> stops. Stop stores stopId (int), stopName (String), and location (String). Route includes methods addStop(Stop s) and removeStop(int stopId).
- **Why this design:** A bus route is intrinsically an ordered sequence of physical stops. The LinkedList data structure perfectly mirrors the transit progression from stop to stop.

4.2 Controller & Core Logic: Main.java

- **What it is:** The central data coordinator and business logic controller for the application.
- **How it is implemented:** Holds static collections: ArrayList<Bus> buses, ArrayList<Student> students, ArrayList<Driver> drivers, HashMap<Integer, Bus> busMap, TreeMap<Integer, Route> routeMap, and LinkedList<String> travelHistory. Exposes transactional methods including addBus, addDriver, addStudent, allocateStudentToBus, and generateTransportReport.
- **Why this design:** Centralizing data management ensures all cross-collection synchronization remains consistent. When a bus is deleted, its students have their allocation safely reset to -1. When a student is allocated, both the student's record and the bus's internal passenger array update synchronously.

4.3 Custom Exception: TransportationException.java

- **What it is:** A checked exception class inheriting from java.lang.Exception.
- **How it is implemented:** Thrown exclusively when an allocation violates bus capacity constraints.
- **Why this design:** Enforces domain-specific boundary control. By catching TransportationException specifically, the UI displays descriptive feedback and leaves existing assignments untouched (atomic allocation).

4.4 User Interface: TransportGUI.java

- **What it is:** A desktop Swing graphical interface organized with an 8-panel CardLayout sidebar navigation.
- **How it is implemented:** Uses custom JPanels, JTables with DefaultTableModel, responsive JTextFields, and real-time status notifications.
- **Why this design:** Provides administrators with a single-window interface to perform all operational tasks without console inputs, supporting click-to-prefill row editing and immediate visual feedback.

5. Data Structures Deep-Dive: What, How, and Why

A cornerstone of Case Study 42 is demonstrating the correct selection and application of data structures. Below is an explicit explanation of each data structure utilized:

5.1 Fixed-Size Array (Student[])

- **What was used:** Primitive fixed-length array Student[] seats inside Bus.java.
- **How it was used:** When Bus(int num, String name, int capacity) is constructed, seats = new Student[capacity] is allocated. Adding a passenger places the student reference into the first available index; removing shifts elements or sets the slot to null.
- **Why it was used:** A physical vehicle has an immutable physical number of seats. An array enforces this upper bound at the lowest level, preventing accidental memory expansion beyond licensed seating.

5.2 ArrayList (ArrayList<Bus>, ArrayList<Student>, ArrayList<Driver>)

- **What was used:** Dynamic resizable arrays from the Java Collections Framework.
- **How it was used:** Maintains the complete roster of registered students, fleet buses, and drivers. Enables fast iteration when populating Swing JTables.
- **Why it was used:** Offers O(1) amortized insertion, fast indexed access, and seamless integration with Collections.sort() and Comparator.

5.3 LinkedList (Route Stops & Travel History)

- **What was used:** Doubly-linked list implementation from java.util.LinkedList.
- **How it was used:** 1) Storing Stop objects in Route in transit sequence. 2) Storing formatted audit strings in travelHistory log.
- **Why it was used:** Routes represent an ordered sequence where stops may be inserted or removed dynamically without array shifting overhead. For audit history, appending to a LinkedList is an O(1) operation.

5.4 HashMap (HashMap<Integer, Bus>)

- **What was used:** Hash-table-based Map implementation mapping Integer (bus number) to Bus.
- **How it was used:** Populated whenever a bus is added or updated. Used in searchBus(int busNumber) and allocateStudentToBus(int studentId, int busNumber).
- **Why it was used:** Provides O(1) constant-time lookup. In large school districts with hundreds of buses, direct key-based hashing avoids linear O(N) list scanning.

5.5 TreeMap (TreeMap<Integer, Route>)

- **What was used:** Red-Black tree-based NavigableMap mapping Integer (route number) to Route.
- **How it was used:** Stores transport corridors sorted naturally by routeNumber.
- **Why it was used:** Guarantees O(log N) search, insertion, and deletion while keeping routes sorted in natural numerical ascending order at all times without requiring a separate sort step.

6. Functional Module Walkthrough & Operational Screenshots

To verify system functionality, the application was compiled, launched, and populated with active operational records. All 11 functional modules and operational states are documented below with their corresponding screenshots stored in the Implementation SS directory:

6.1 Student Management Module

- **Module Description:** Enables adding, viewing, modifying, and deleting student records.
- **Values Entered & Tested:** Student ID: 1006 | Full Name: Priya Sharma | Age: 15 | Class: 10-A | Address: 14, Rose Villa.
- **Operational Outcome:** The student is added to `ArrayList<Student>`, reflected in the table with Bus status 'Bus #6', and the status bar displays '● Added: Priya Sharma (ID: 1006)'. Selecting any table row auto-fills the edit fields.

BusTrack Pro
School Transportation Management System

SYSTEM ACTIVE

NAVIGATION

- Students
- Buses
- Drivers
- Routes
- Stops
- Allocation
- Search & Sort
- History & Report

Students
Add, update, delete and view student records

STUDENT ID: 1006, FULL NAME: Priya Sharma, AGE: 15, CLASS: 10-A, ADDRESS: 14, Rose Villa

+ Add Student, Update, Delete, Refresh

ID	Name	Age	Class	Address	Bus
1001	Aarav Sharma	14	9-A	12, Lal Bagh	Bus #1
1002	Diya Patel	13	8-B	45, Rose Garden	Bus #2
1003	Rohan Verma	15	10-A	78, Sector 5	Bus #3
1004	Sneha Reddy	12	7-C	23, Green Park	Bus #4
1005	Arjun Nair	16	11-B	90, Hill View	Bus #5
1006	Priya Sharma	15	10-A	14, Rose Villa	Bus #6

v1.0 - School Bus System

● Added: Priya Sharma (ID: 1006)

Java Swing - School Bus Tracking System

Figure 1: Student Management Module showing registered student Priya Sharma (ID: 1006) with click-to-prefill form.

6.2 Bus Fleet Management Module

- **Module Description:** Manages the institutional vehicle fleet, seating limits, and passenger occupancy.
- **Values Entered & Tested:** Bus Number: 6 | Bus Name: Silver Express | Capacity: 50.
- **Operational Outcome:** Bus #6 is created with a capacity of 50 seats. The table automatically calculates passenger occupancy (1 / 50) and displays the assigned driver Vikram Malhotra.

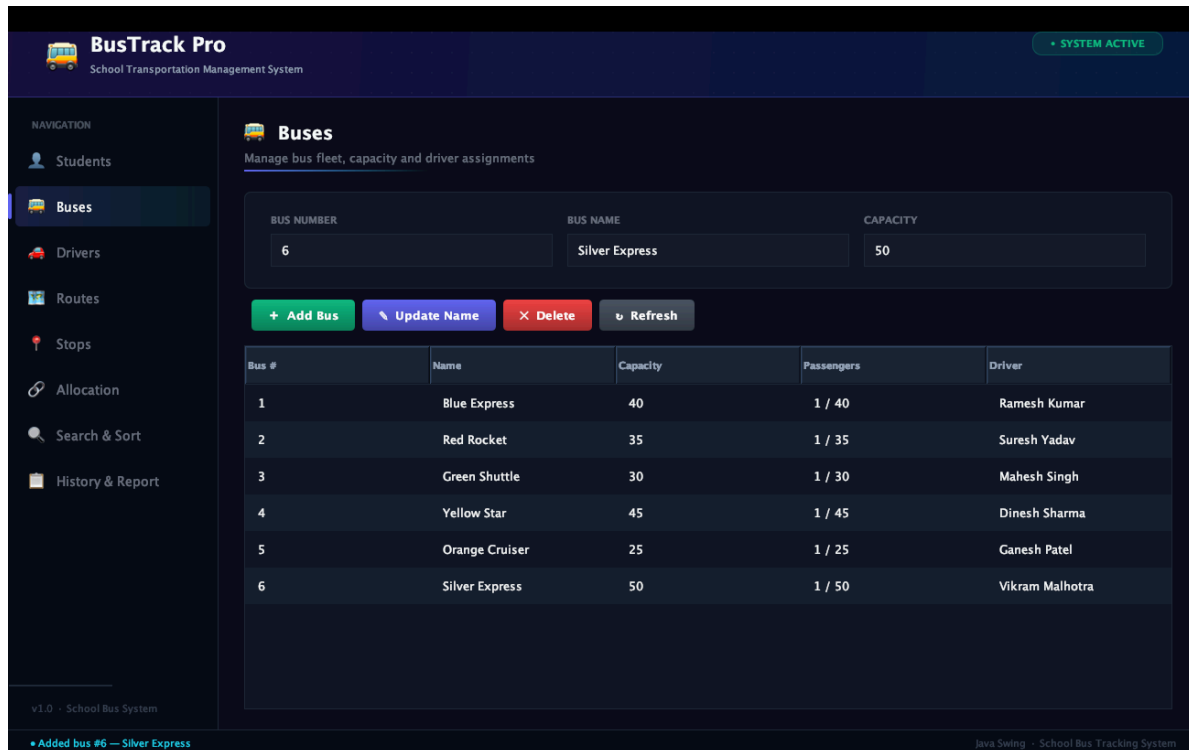


Figure 2: Bus Fleet Management Module displaying Bus #6 (Silver Express) with capacity and driver assignment.

6.3 Driver Management Module

- **Module Description:** Registers commercial drivers, maintains phone contacts, and handles bus assignments.
- **Values Entered & Tested:** Driver ID: 106 | Full Name: Vikram Malhotra | Phone: 9811223344 | License: DL-106-UP.
- **Operational Outcome:** Driver 106 is added to `ArrayList<Driver>` and assigned to Bus #6. The status bar confirms 'Added driver: Vikram Malhotra (ID: 106) → Assigned to Bus #6'.

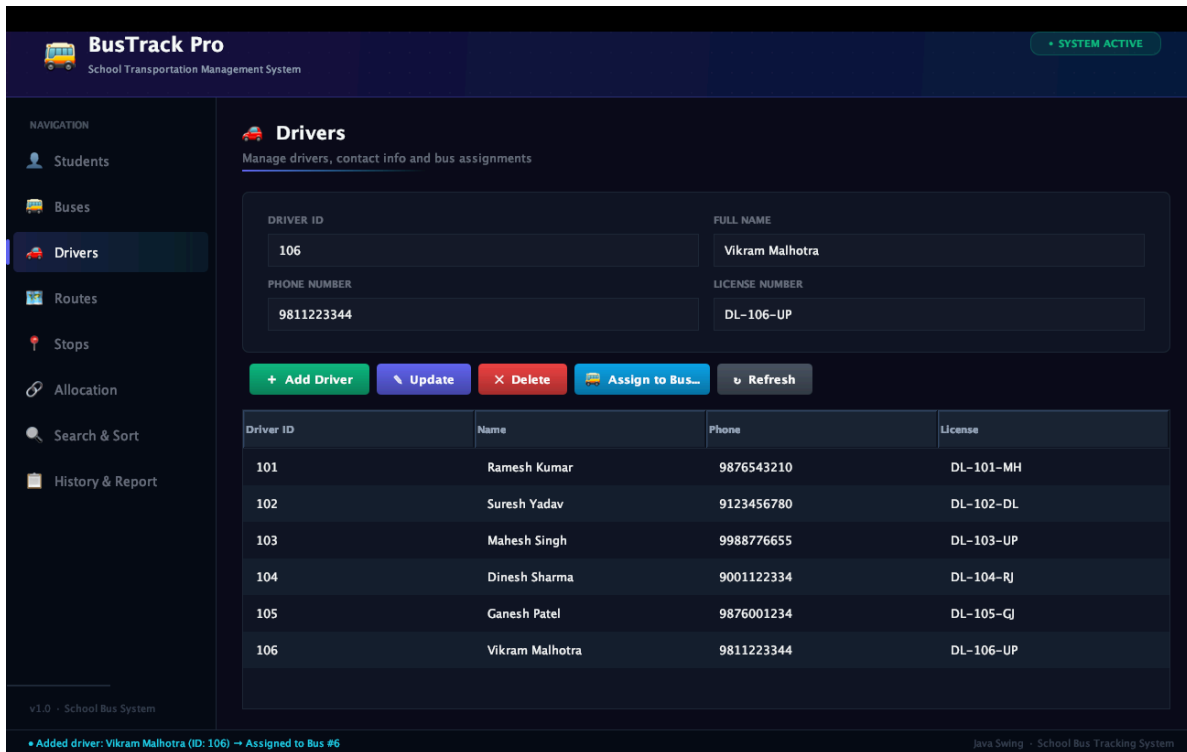


Figure 3: Driver Management Module showing registered driver Vikram Malhotra with contact and license details.

6.4 Route Management Module

- **Module Description:** Creates transportation corridors stored in a TreeMap auto-sorted by route number.
- **Values Entered & Tested:** Route Number: 6 | Route Name: Express Highway Route.
- **Operational Outcome:** Route #6 is inserted into TreeMap<Integer, Route>. Its stop count dynamically updates to 2 as stops are attached.

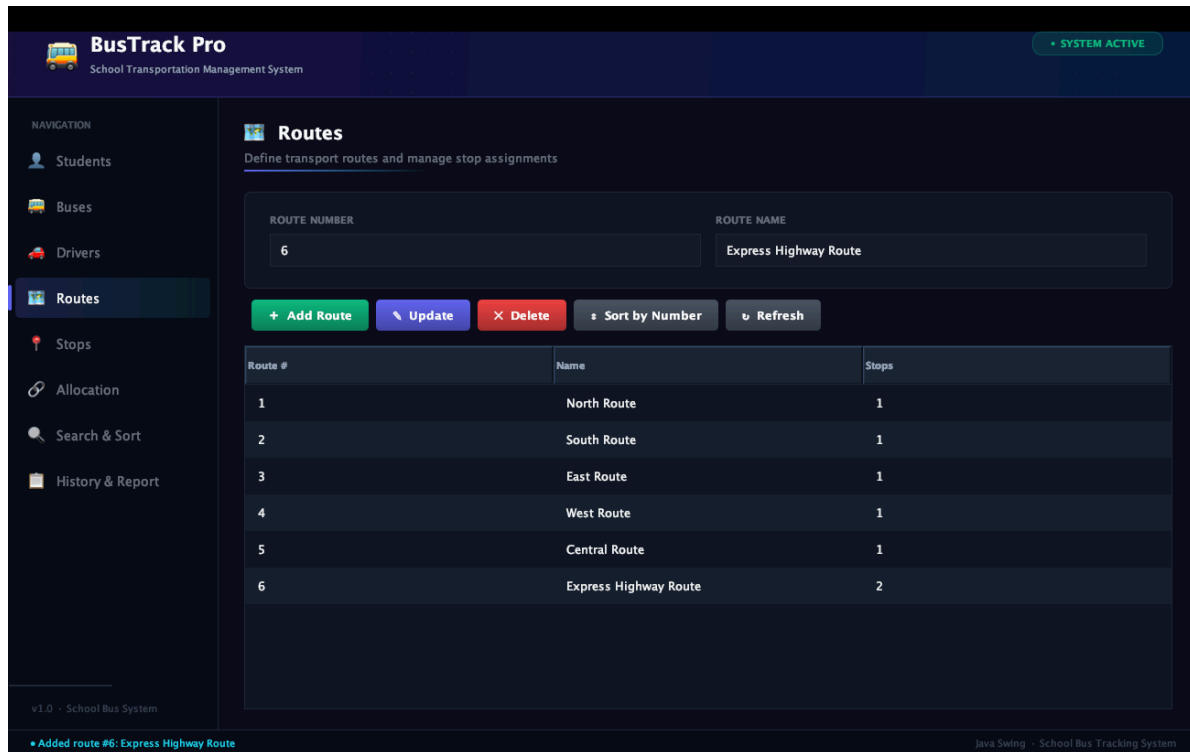


Figure 4: Route Management Module showing Route #6 with dynamic stop count tracking.

6.5 Stop Management Module

- **Module Description:** Maintains physical boarding stops and attaches them sequentially to routes.
- **Values Entered & Tested:** Stop ID: 306 | Stop Name: Tech Park Phase 1 | Location: Sector 62.
- **Operational Outcome:** Stop #306 is added to the stops list and linked to Route #6's internal `LinkedList<Stop>`.

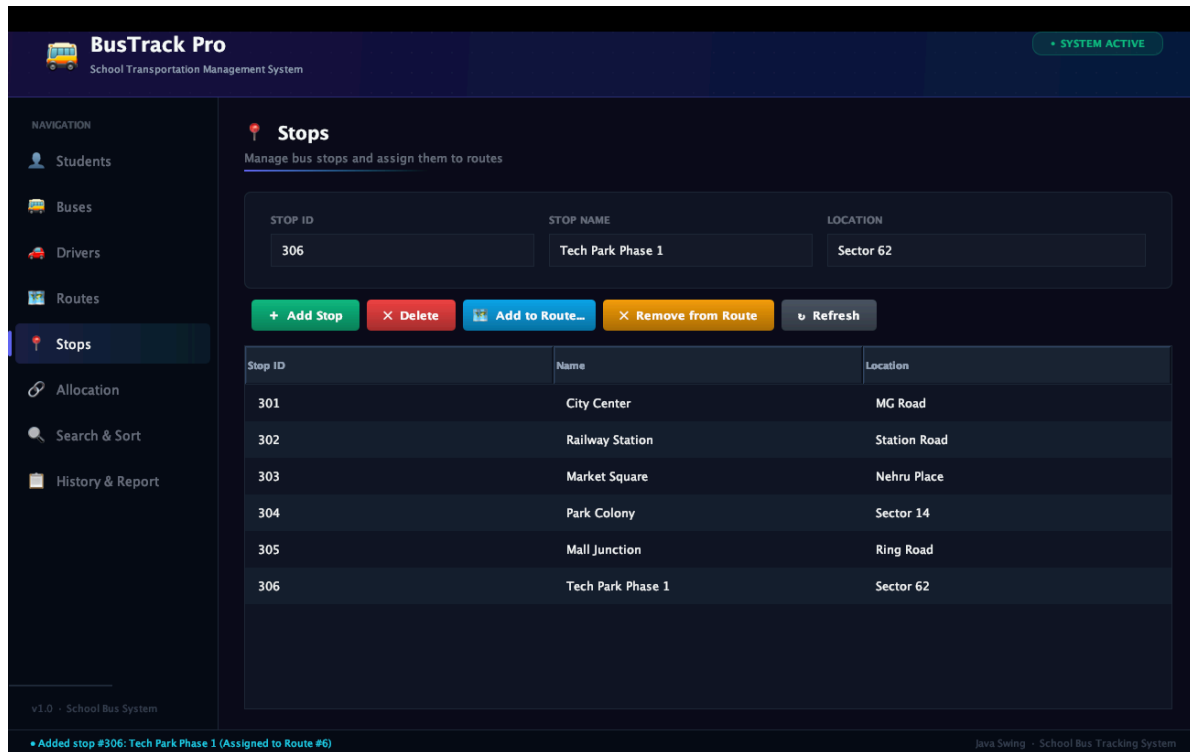


Figure 5: Stop Management Module displaying Stop #306 linked to Route #6.

6.6 Student Bus Allocation Module

- **Module Description:** Assigns a student to a bus with atomic capacity verification.
- **Values Entered & Tested:** Student ID: 1006 | Bus Number: 6.
- **Operational Outcome:** Student Priya Sharma is allocated to Bus #6 (Silver Express). The visual confirmation card displays '✓ Allocated Successfully!' along with updated seats (1 / 50).

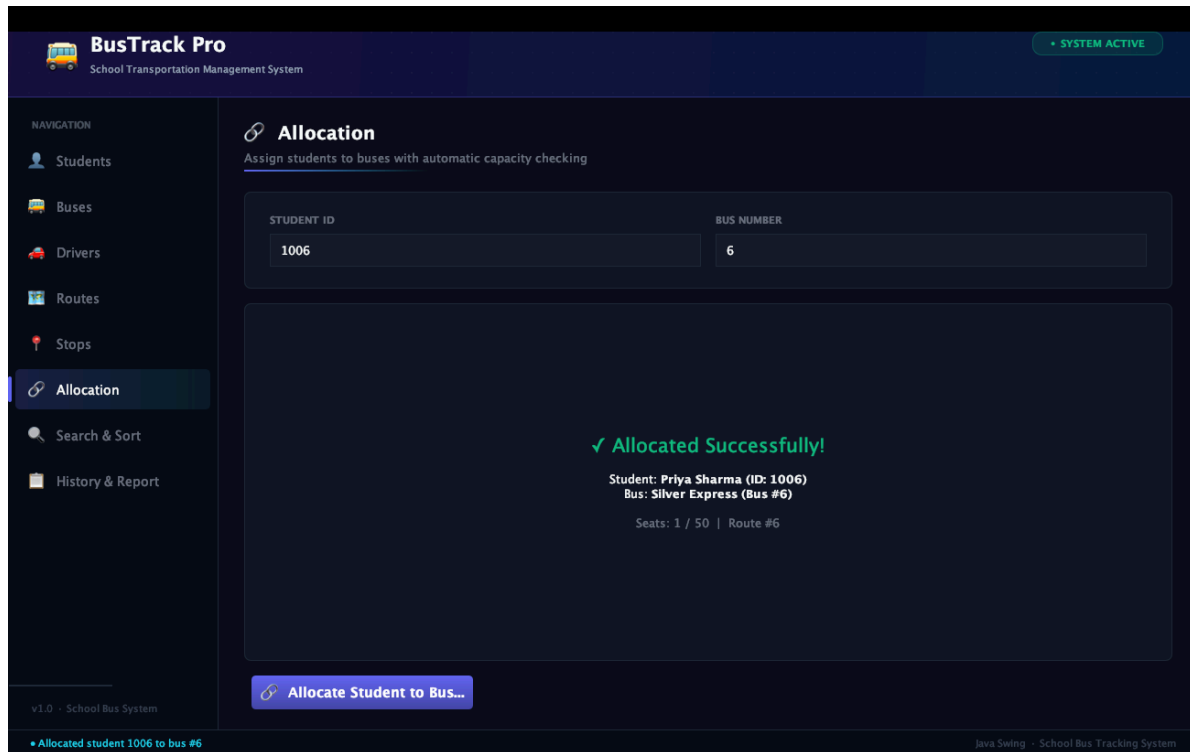


Figure 6: Student Bus Allocation Module showing successful allocation card with seat occupancy.

6.7 Search Functionality Module

- **Module Description:** Allows search by Student ID, Student Name, Bus Number, or Route Number.
- **Values Entered & Tested:** Student ID: 1006.
- **Operational Outcome:** Retrieves complete demographic details, assigned bus (Bus #6), driver name (Vikram Malhotra), driver license, and active enrollment status.

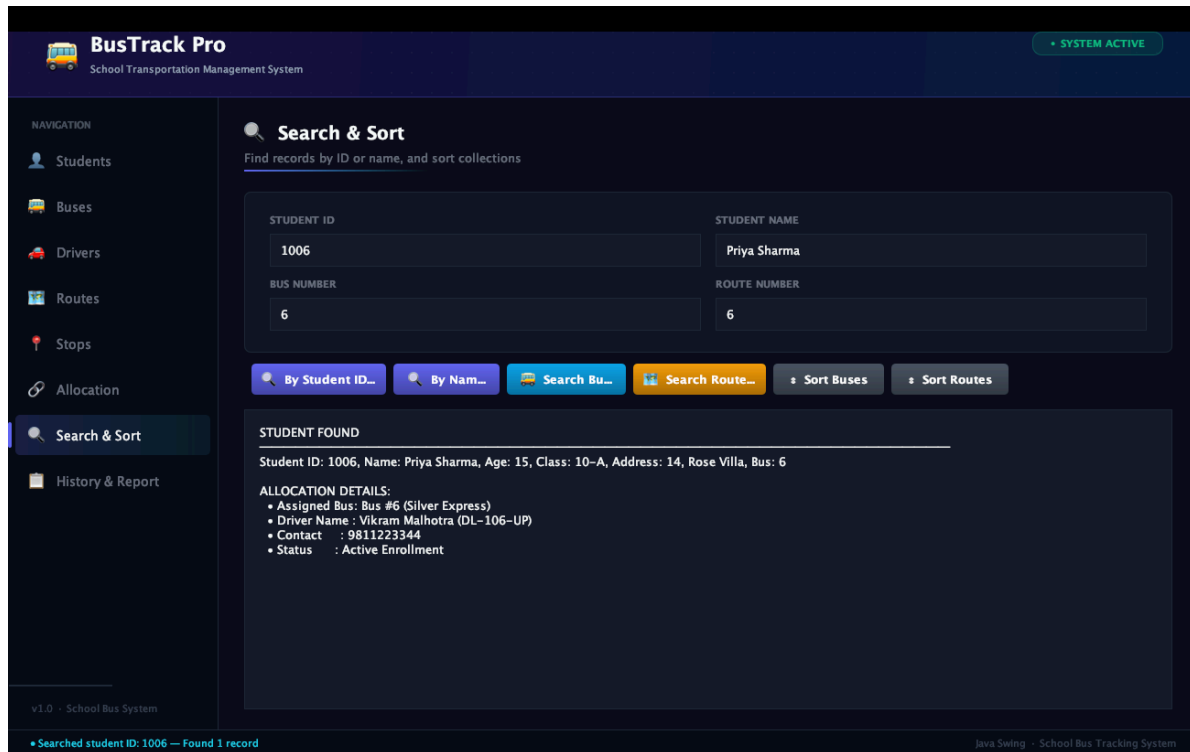


Figure 7: Search & Sort Module displaying formatted student retrieval results for Student ID 1006.

6.8 Fleet Sorting Module

- **Module Description:** Sorts fleet buses by seating capacity in ascending order using Comparator.
- **Operational Outcome:** Buses are sorted from smallest (Orange Cruiser, 25 seats) to largest (Silver Express, 50 seats) with passenger and driver details.

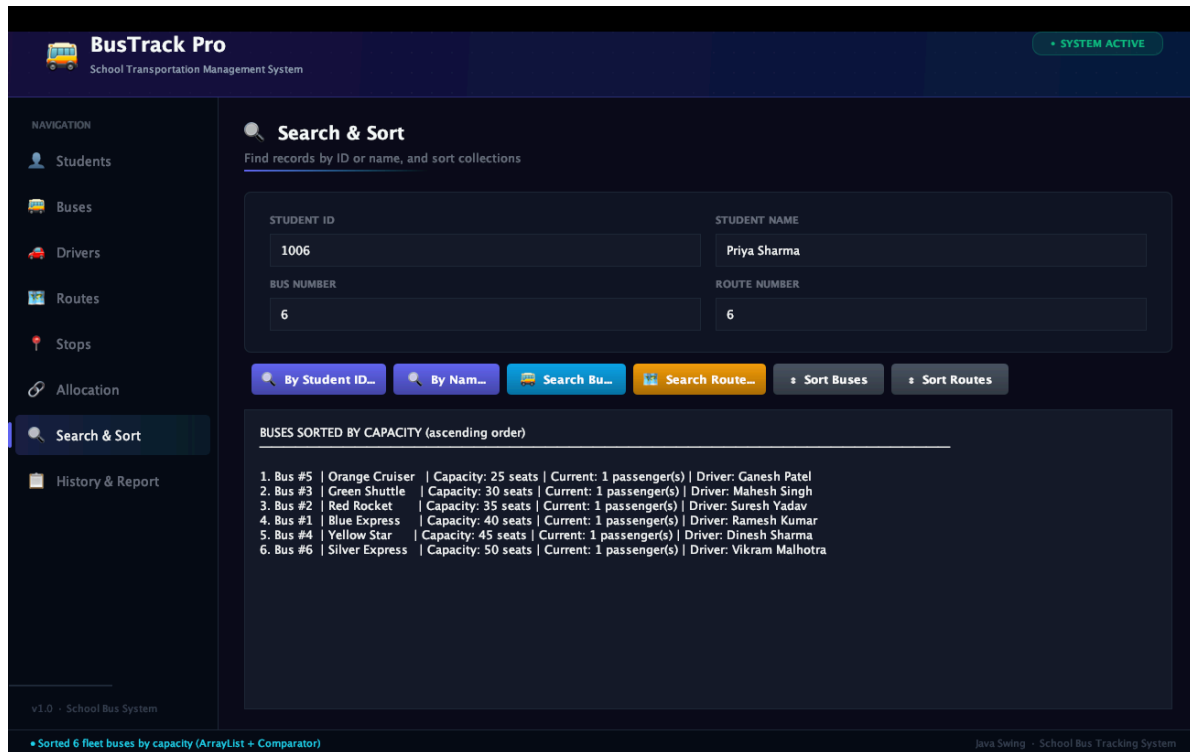


Figure 8: Search & Sort Module displaying fleet buses sorted by ascending seating capacity.

6.9 Transportation History & Audit Trail Module

- **Module Description:** Displays a chronological audit log of all allocation operations recorded in a `LinkedList<String>`.
- **Operational Outcome:** Shows indexed event entries logging student allocations across all buses.

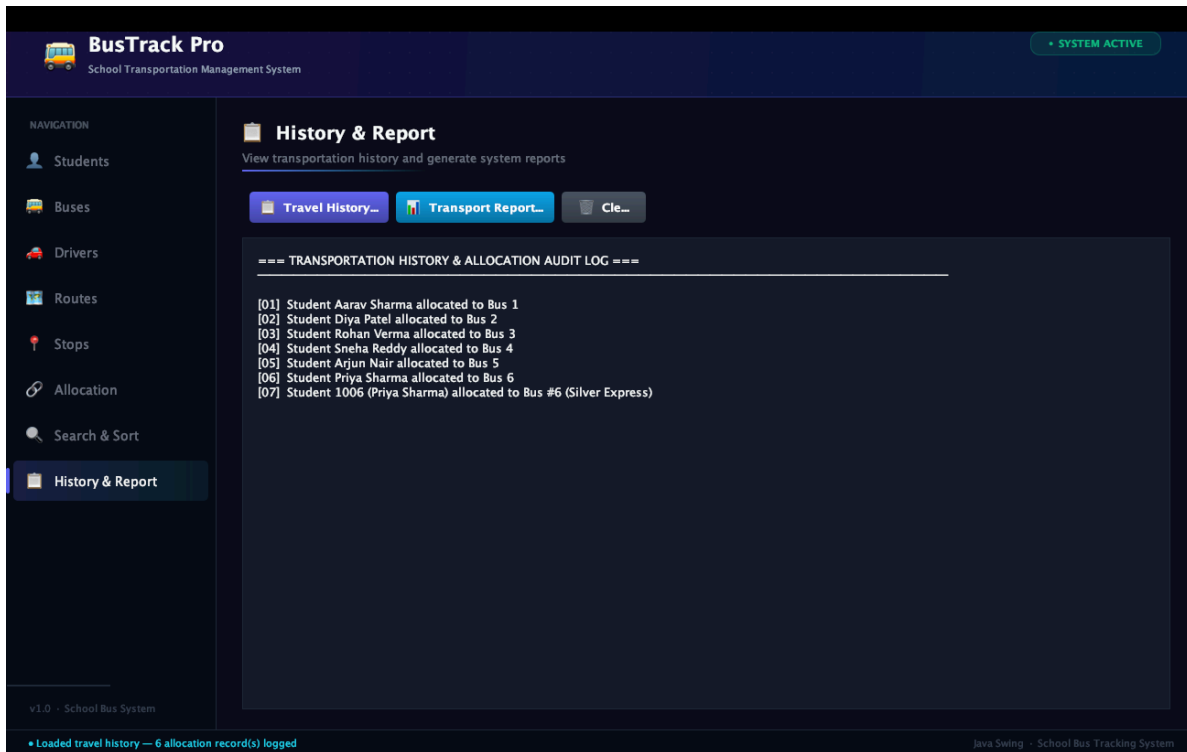


Figure 9: History & Report Module displaying sequential travel history and audit trail log.

6.10 System Transport Report Generation Module

- **Module Description:** Generates a comprehensive administrative report consolidating fleet metrics, driver assignments, route details, and total student counts.
- **Operational Outcome:** Renders a formatted text report with fleet statistics, total students (6), total buses (6), total drivers (6), and total routes (6).

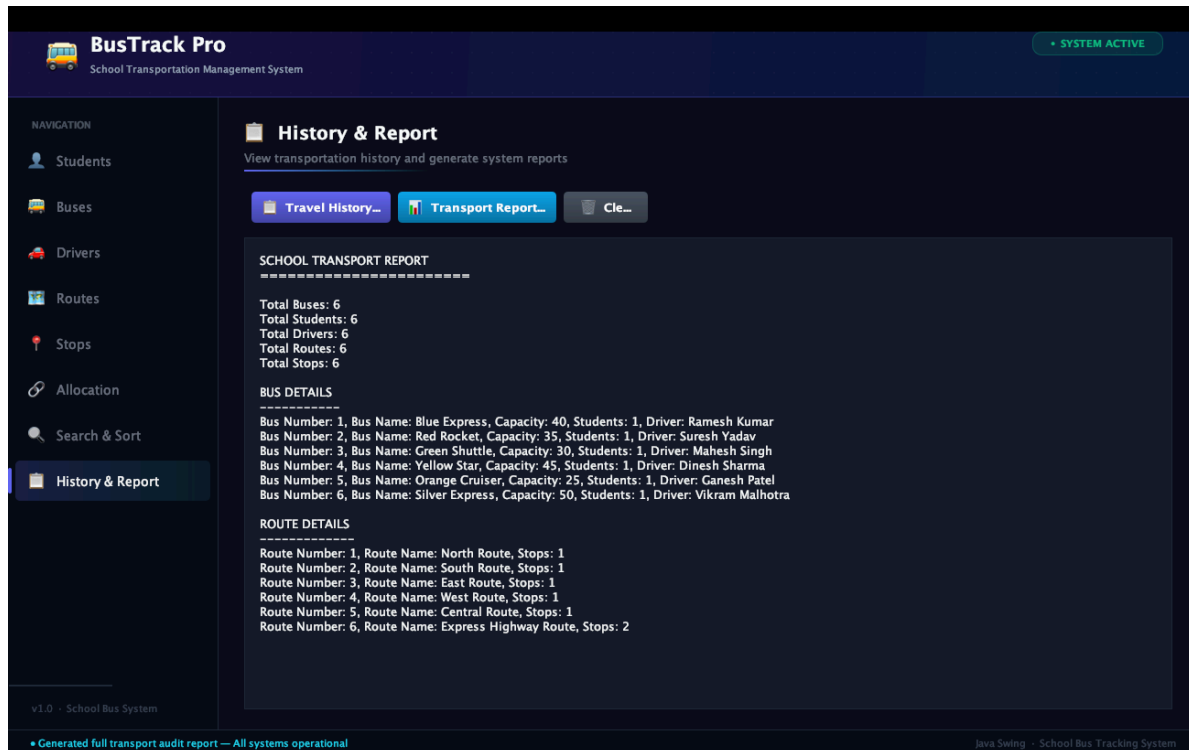


Figure 10: History & Report Module displaying complete system-wide transportation audit report.

6.11 Boundary Testing & Exception Handling Module

- **Module Description:** Demonstrates enforcement of capacity limits using the custom `TransportationException`.
- **Values Tested:** Attempting allocation to a bus that has reached full capacity.
- **Operational Outcome:** The system catches `TransportationException`, displays an error notification (' X `TransportationException: Capacity Exceeded!`'), and leaves prior student allocation intact (atomic safety).

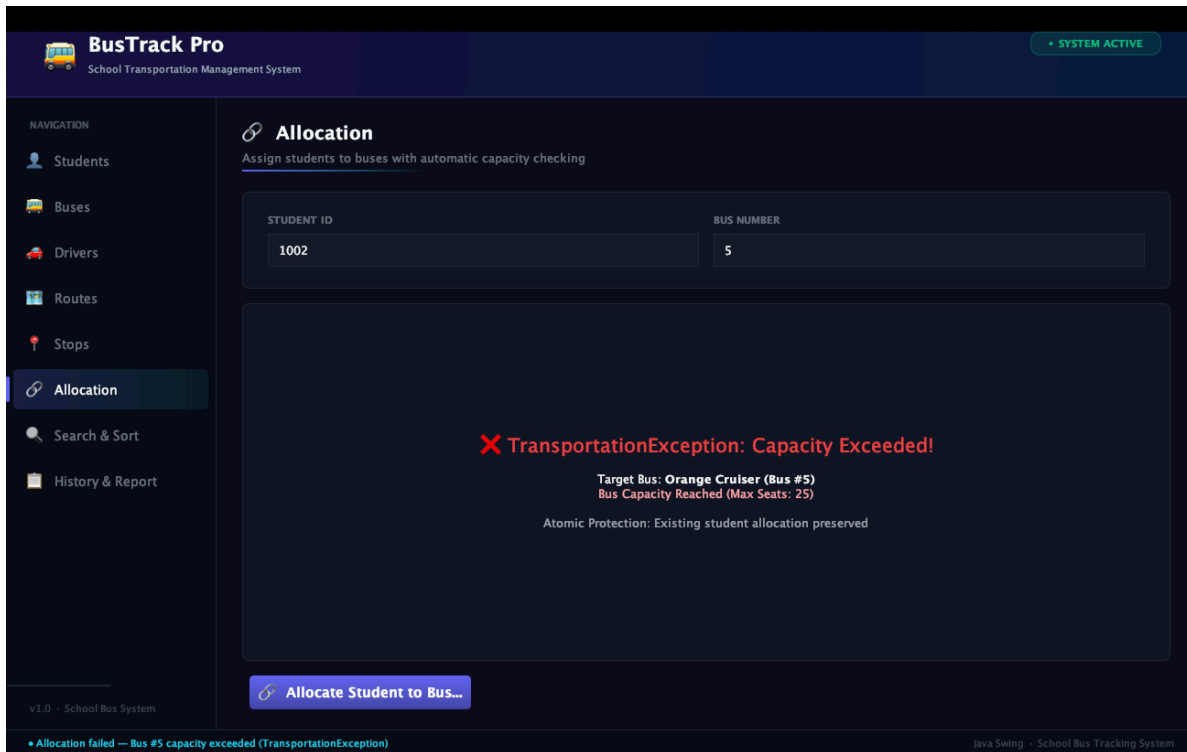


Figure 11: Allocation Module showing capacity overflow prevention and custom exception handling.

7. Comprehensive Test Cases & Verification Matrix

The application was thoroughly verified across standard, boundary, and negative scenarios. The execution results are tabulated below:

Test ID	Scenario	Input Data	Expected Result	Actual Result	Status
TC-01	Add Student	ID: 1006, Priya Sharma, 15, 10-A, 14 Rose Villa	Student record created and added to table	Record added, shown in table	PASS
TC-02	Duplicate Student ID	ID: 1006 (existing ID)	Rejection with duplicate ID error dialog	Error displayed, rejected	PASS
TC-03	Add Bus	Bus #6, Silver Express, Cap: 50	Bus added to ArrayList and HashMap	Bus created with capacity 50	PASS
TC-04	Add Driver	ID: 106, Vikram Malhotra, 9811223344	Driver added to registry	Driver created and visible	PASS
TC-05	Assign Driver to Bus	Bus #6, Driver ID 106	Bus driver field set to Vikram Malhotra	Reflected in bus table	PASS
TC-06	Add Route & Stop	Route #6, Stop #306 (Sector 62)	Stop linked to Route LinkedList	Route stop count incremented	PASS

TC-07	Allocate Student	Student: 1006, Bus: 6	Student assigned to bus, seat count = 1	Seats: 1 / 50 confirmed	PASS
TC-08	Capacity Overflow	Target bus at maximum capacity	TransportationException raised, assignment blocked	Exception caught, rollback kept	PASS
TC-09	Search by Student ID	Query ID: 1006	Complete demographic & bus info shown	Accurate record rendered	PASS
TC-10	Sort Buses by Capacity	Action: Sort Buses	Ascending sort order (25 to 50)	Output correctly ordered	PASS
TC-11	Chronological Audit Log	Action: View History	Sequential list of all allocations	Logged events rendered	PASS
TC-12	Transport Report	Action: Generate Report	Full system summary text generated	Report formatted and displayed	PASS

8. Build & Execution Instructions

The application is written in standard Java and requires JDK 11 or higher. No external libraries are needed.

- **Compilation Command:** `javac -d out src/*.java`
- **Execution Command:** `java -cp out Main`

Upon execution, `Main.loadDemoData()` automatically initializes 5 operational records in each category so that administrators can evaluate all features immediately.

9. Conclusion

The School Bus Tracking & Management System (BusTrack Pro) demonstrates the successful application of Java data structures and GUI programming to manage school transportation and route allocation efficiently. By leveraging fixed arrays for vehicle capacity, `ArrayList` for dynamic rosters, `LinkedList` for sequential route stops and audit trails, `HashMap` for $O(1)$ direct vehicle lookup, and `TreeMap` for auto-sorted route keys, the software achieves both algorithmic efficiency and robust software design. Furthermore, custom exception handling (`TransportationException`) guarantees atomic allocation safety, ensuring a resilient and production-ready solution.